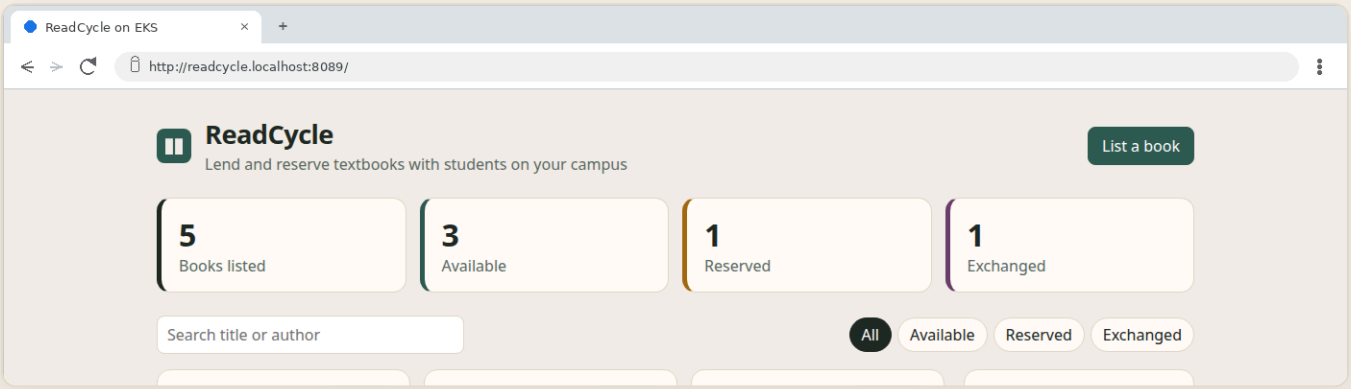
DEVOPS COURSE · SESSION 21 · FINAL CAPSTONE

ReadCycle

A campus textbook exchange, and the delivery system that ships it: from one `git push` to a tested, scanned, monitored Kubernetes deployment.

Ratnesh Vaibhav · Roll No 24bcs10413 · 16 October 2026



11 tests pytest, 95 % coverage	**11 CI jobs** test → scan → build → push → deploy	**0 HIGH / CRIT.** 8 security reports, one gate	**27 AWS res.** VPC + EKS by Terraform, destroyed	**5 min 49 s** git push → live, via Argo CD	**46 / 46** self-checks against GRADING.md

REPOSITORY	github.com/RatneshVaibhav/readcycle
DEMO VIDEO	Watch on Google Drive (4 min 22 s, narrated, with captions; a copy is in `docs/ReadCycle-Demo.mp4`)
EVIDENCE	docs/EVIDENCE.md: all 53 screenshots, item by item against the rubric
PIPELINE	github.com/RatneshVaibhav/readcycle/actions

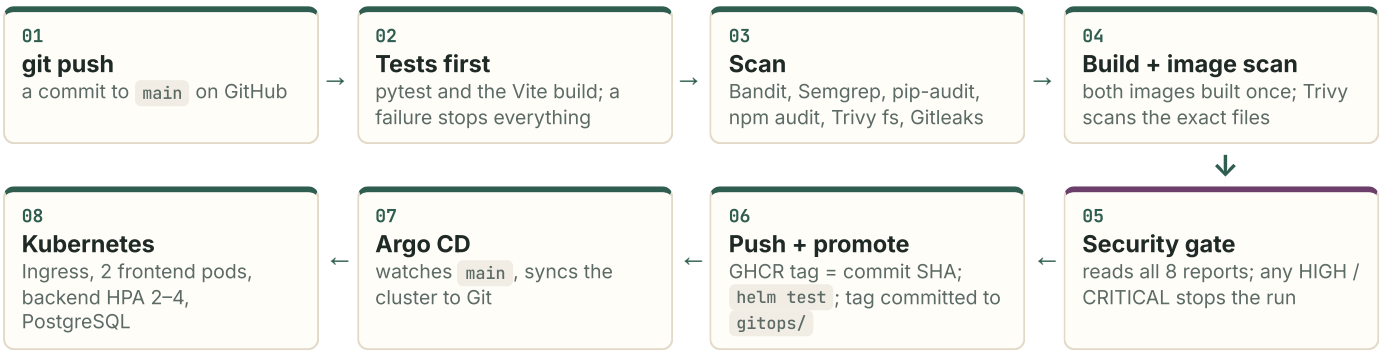
The rubric, and where this report proves each module

Module	What was delivered	Pts	Page
M1	Application: FastAPI + React + PostgreSQL, 8 REST endpoints, Alembic migration	10	3
M2	Testing: 11 pytest tests, 95 % coverage, in-memory SQLite test database	10	3
M3	Git and GitHub: public repository, 50 commits, `.gitignore` verified	5	4
M4	Docker: multi-stage frontend, non-root images, Compose with health checks	10	4
M5	CI/CD: 11-job GitHub Actions pipeline, GHCR images tagged with the commit SHA	15	5
M6	DevSecOps: Trivy on both images + SAST, SCA, secrets; one gate over 8 reports	5	5
M7	Terraform on AWS: VPC, 2 public + 2 private subnets, NAT, EKS + node group; destroyed	15	6
M8	Kubernetes + Helm: chart on EKS, 2+ replicas, ClusterIP, Ingress `/` and `/api`, HPA	15	7
M9	Observability: `/metrics`, Prometheus targets UP, Grafana dashboard, alerts	10	8
M10	Documentation and live demo: README, demo script, recorded GitOps rollout	5	9
	Troubleshooting challenge: 3 faults injected through Git, root-caused, reverted		10

Architecture

The app is deliberately small. The project is the delivery system around it: how one commit travels from the laptop to a monitored Kubernetes cluster, safely and automatically.

A change: from `git push` to production



CI never touches the cluster and **no cloud or cluster credentials are stored in GitHub**. Rolling back is `git revert`.

A request, and who watches it

Ingress (ingress-nginx)
`/` → frontend: nginx serving the React build, 2 pods. `/api` → backend: FastAPI, autoscaled 2–4 pods

PostgreSQL 17
StatefulSet with a persistent volume (gp3 EBS on AWS); password from a Secret that is never in Git

Prometheus + Grafana
ServiceMonitor scrapes `/metrics` every 15 s; 4 alert rules; 12-panel dashboard

It ran in three places: on **AWS EKS** in Sydney (built by Terraform, destroyed after testing), on a **local kind cluster** deployed by Argo CD from GitHub (the live demo and the troubleshooting challenge, at no cost), and with **Docker Compose** for local development.

Technologies

Area	Tools
Application	React 19 + Vite 8 · Python 3.12, FastAPI, SQLAlchemy 2, Alembic, Pydantic 2 · PostgreSQL 17 · pytest
Delivery	Docker multi-stage builds, Docker Compose, GitHub Actions, GitHub Container Registry, Argo CD 3.5
Security	Bandit, Semgrep, pip-audit, npm audit, Trivy, Gitleaks, a security gate, Pod Security
Infrastructure	Terraform (AWS provider 6), VPC, EKS 1.37, IAM, EKS Pod Identity, EBS CSI driver
Kubernetes	Helm 4, ingress-nginx, HPA + metrics-server, kind, kube-prometheus-stack

Cost: $0 billed
GitHub, Actions, GHCR and the local cluster are free. The AWS session (EKS, NAT gateway, one node, about 1 h 20 min) used about $0.40 of Free Tier credits on the AWS Free plan. Everything was destroyed afterwards, and the Terraform user and its access key were deleted (`docs/TEARDOWN.md`).

No secrets anywhere
Gitleaks scans the whole Git history on every run. The database password lives in a git-ignored `.env` and in a Kubernetes Secret; Grafana's is random; the pipeline uses only GitHub's own short-lived token.

How the evidence was made
Terminal screenshots are rendered from recorded sessions, each command with its real output (`docs/evidence/transcripts/`). Browser screenshots are headless-Chrome captures with the URL bar. AWS Console screenshots were taken by me in my AWS account. In this report, tall screenshots are cropped to the lines that matter; each caption names the full file in `docs/evidence/screenshots/` . `scripts/check-submission.sh` re-checks every scriptable rubric item: **46 passed, 0 failed**.

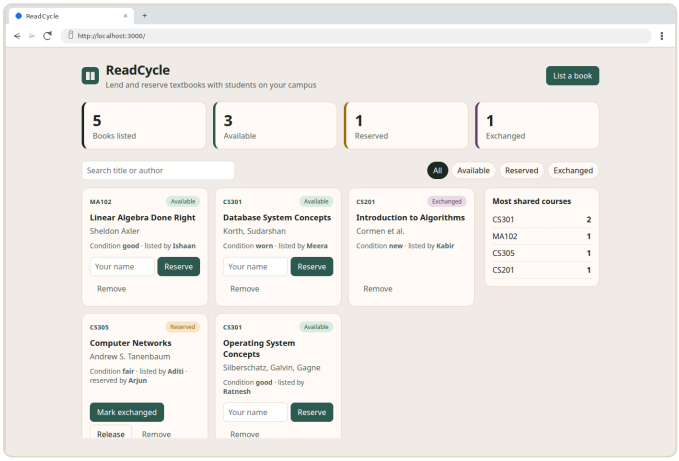
M1 Application

10 pts

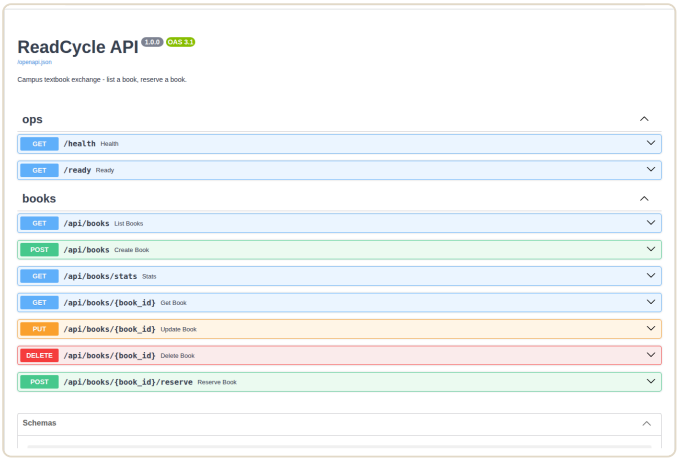
Students list course books they no longer need, other students reserve them, and the owner marks a book as exchanged once it changes hands.

Endpoint	Purpose
`GET /api/books?status=&q=&course=`	list and search
`POST /api/books`	list a book (201)
`GET · PUT · DELETE /api/books/{id}`	read, update, remove
`POST /api/books/{id}/reserve`	reserve; 409 if taken
`GET /api/books/stats`	totals, top courses
`GET /api/version`	running commit (UI footer)
`/health · /ready · /metrics`	probes, Prometheus

Rubric criterion	
FastAPI starts, `/health` responds (`m1-api_1.png`)	✓
≥ 4 REST endpoints, GET/POST/PUT/DELETE, each with its real status code: 201, 200, 409, 204, 404, 422	✓
PostgreSQL table by Alembic: `0001_create_books.py`, "Running upgrade → 0001" (page 4)	✓
React frontend renders and calls the API; responsive layout	✓



The React UI on Docker Compose, localhost:3000. `web-m1-m4-app-localhost-3000.png`



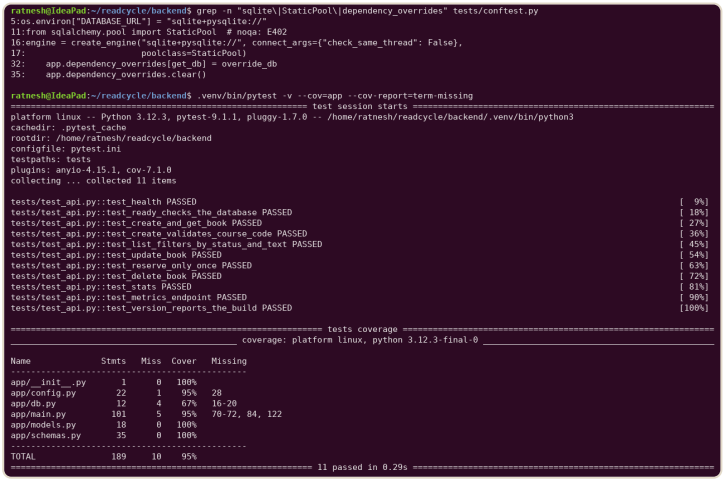
The API: every endpoint in the FastAPI docs. `web-m1-swagger-api-docs.png`

M2 Testing

10 pts

Eleven tests cover health, readiness, create and get, validation, search and filters, update, reserve-only-once, delete, stats, `/metrics` and the version endpoint. They run against an **in-memory SQLite** database, so they never touch real data.

Rubric criterion	
`pytest` runs without errors; ≥ 5 tests on ≥ 3 endpoints: **11 passed, 95 % coverage**	✓
A test database, not production: `sqlite+pysqlite://` in `conftest.py`	✓
`pytest.ini` and `conftest.py` present	✓
Tests gate the pipeline: every later job `needs` them	✓



```
ratnesh@IdeaPad:~/readcycle/backend$ grep -n "sqlite\|StaticPool\|dependency_overrides" tests/conftest.py
5:os.environ["DATABASE_URL"] = "sqlite+pysqlite://"
11:from sqlalchemy.pool import StaticPool  # noqa: E402
16:engine = create_engine("sqlite+pysqlite://", connect_args={"check_same_thread": False},
17:                       poolclass=StaticPool)
32:    app.dependency_overrides[get_db] = override_db
35:    app.dependency_overrides.clear()

ratnesh@IdeaPad:~/readcycle/backend$ .venv/bin/pytest -v --cov=app --cov-report=term-missing
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.7.0 -- /home/ratnesh/readcycle/backend/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/ratnesh/readcycle/backend
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1, cov-7.1.0
collecting ... collected 11 items

tests/test_api.py::test_health PASSED                                    [  9%]
tests/test_api.py::test_ready_checks_the_database PASSED                 [ 18%]
tests/test_api.py::test_create_and_get_book PASSED                       [ 27%]
tests/test_api.py::test_create_validates_course_code PASSED              [ 36%]
tests/test_api.py::test_list_filters_by_status_and_text PASSED           [ 45%]
tests/test_api.py::test_update_book PASSED                               [ 54%]
tests/test_api.py::test_reserve_only_once PASSED                         [ 63%]
tests/test_api.py::test_delete_book PASSED                               [ 72%]
tests/test_api.py::test_stats PASSED                                     [ 81%]
tests/test_api.py::test_metrics_endpoint PASSED                          [ 90%]
tests/test_api.py::test_version_reports_the_build PASSED                 [100%]

---------- coverage: platform linux, python 3.12.3-final-0 -----------

Name             Stmts   Miss  Cover   Missing
----------------------------------------------
app/__init__.py      1      0   100%
app/config.py       22      1    95%   28
app/db.py           12      4    67%   16-20
app/main.py        101      5    95%   70-72, 84, 122
app/models.py       18      0   100%
app/schemas.py      35      0   100%
----------------------------------------------
TOTAL              189     10    95%
============================== 11 passed in 0.29s ==============================
```

The SQLite test database in `conftest.py`, then `pytest -v`: 11 passed, 95 % coverage. `m2-pytest.png`

M3 Git and GitHub — 5 pts

Rubric criterion		Proof
Public GitHub repository	✓	github.com/RatneshVaibhav/readcycle
≥ 10 commits with meaningful messages	✓	50 commits today, each describing one change (17 when this was recorded)
`.gitignore` excludes `.env`, `__pycache__`, `node_modules`, `.venv`	✓	`git check-ignore` for each, below

```
ratnesh@IdeaPad:~/readcycle$ git remote -v | head -1; git log --oneline | wc -l | sed "s/^/commits on main: /"
origin  https://github.com/RatneshVaibhav/readcycle.git (fetch)
commits on main: 17

ratnesh@IdeaPad:~/readcycle$ git log --format="%h  %<(16,trunc)%an %s" | head -24
0881437  Ratnesh Vaibhav  Add CI/CD and security evidence from GitHub
8f3640e  github-actions.. Deploy ecfd0c2 to dev [skip ci]
ecfd0c2  Ratnesh Vaibhav  Add local cluster and monitoring scripts, and Kubernetes and monitoring evidence
90f4b9d  github-actions.. Deploy a042073 to dev [skip ci]
a042073  Ratnesh Vaibhav  Add evidence tooling and the first evidence: tests, Compose, API, metrics
6d36399  Ratnesh Vaibhav  Let pytest -v list every test
4ccc035  Ratnesh Vaibhav  Make the Terraform safe and cheap for a real AWS Free plan account
380d194  github-actions.. Deploy c707b5f to dev [skip ci]
c707b5f  Ratnesh Vaibhav  Log in to GHCR before the deploy test pulls the new images
c9fc214  Ratnesh Vaibhav  Rewrite the README and troubleshooting guide for the standalone repository
79366e0  Ratnesh Vaibhav  Route ReadCycle alerts to their own receiver in the monitoring values
6ff37a1  Ratnesh Vaibhav  Deploy through GitOps from this GitHub repository
3cf8607  Ratnesh Vaibhav  Add a .gitignore and keep terraform.tfvars out of Git
1696e2f  Ratnesh Vaibhav  Rename the project from ShelfShare to ReadCycle
555cd84  Ratnesh Vaibhav  Move to the grader's folder layout
81a235d  Ratnesh Vaibhav  Session 21: CI-to-GitOps rollout, troubleshooting challenge, monitoring, documentation
7f170ca  Ratnesh Vaibhav  Session 21: ShelfShare final project - app, containers, Kubernetes, Helm, Terraform, security, GitOps, pipeline

ratnesh@IdeaPad:~/readcycle$ git check-ignore -v .env backend/.venv backend/app/__pycache__ frontend/node_modules terraform/terraform.tfstat
e | sed "s/\t/   ->   /"
.gitignore:3:*.env   ->   .env
.gitignore:29:.venv/   ->   backend/.venv
.gitignore:27:__pycache__/   ->   backend/app/__pycache__
frontend/.gitignore:1:node_modules/   ->   frontend/node_modules
.gitignore:15:*.tfstate   ->   terraform/terraform.tfstate
```

The remote, the commit history (including the pipeline bot's GitOps commits) and the `.gitignore` check. `m3-git-history.png`

M4 Docker — 10 pts

- **Backend**: two stages; dependencies built in a throw-away stage, runtime `python:3.12-alpine` as **uid 10001**; migrations run before the API starts.
- **Frontend**: **multi-stage**, `FROM node:24-alpine AS build` → `FROM nginxinc/nginx-unprivileged` as **uid 101**, no Node at runtime; OS packages patched (M6).
- **Compose**: PostgreSQL with a health check; the backend waits for the database, the frontend for the backend; the password comes from a git-ignored `.env`.

```
SERVICE    IMAGE                       STATUS                  PORTS
backend    readcycle-backend:compose   Up 5 seconds (healthy)  0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
db         postgres:17-alpine          Up 11 seconds (healthy) 5432/tcp
frontend   readcycle-frontend:compose  Up Less than a second   0.0.0.0:3000->8080/tcp, [::]:3000->8080/tcp

ratnesh@IdeaPad:~/readcycle$ docker compose logs backend | grep -E "Running upgrade|Uvicorn running|Application startup"
backend-1  | INFO  [alembic.runtime.migration] Running upgrade  -> 0001, create books table
backend-1  | INFO:     Application startup complete.
backend-1  | INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)

ratnesh@IdeaPad:~/readcycle$ echo "backend runs as: $(docker compose exec backend id)"; echo "frontend runs as: $(docker compose exec fronte
nd id)"
backend runs as: uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)
frontend runs as: uid=101(nginx) gid=101(nginx) groups=101(nginx)

ratnesh@IdeaPad:~/readcycle$ docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}" | grep -E "REPOSITORY|readcycle-(backend|fr
ontend)"
REPOSITORY                                      TAG                                     SIZE
readcycle-frontend                              compose                                 111MB
readcycle-backend                               compose                                 201MB
```

After `docker compose up --build`: three services healthy, the Alembic migration applied, **both containers non-root** (uid 10001, uid 101), both images built. `m4-compose-up.png`

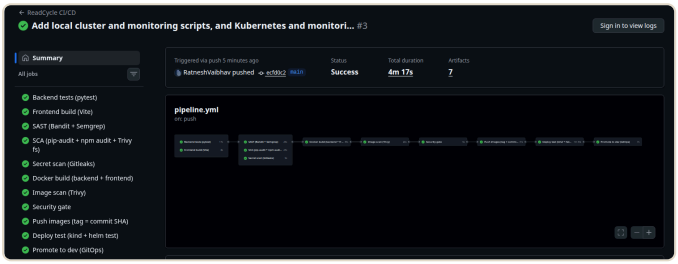
Rubric criterion	
Both Dockerfiles build; the frontend is multi-stage (Node build + nginx runtime)	✓
Images run as non-root	✓
`docker compose up --build` starts frontend, backend and PostgreSQL; the UI works at localhost:3000 (page 3)	✓

M5 CI/CD

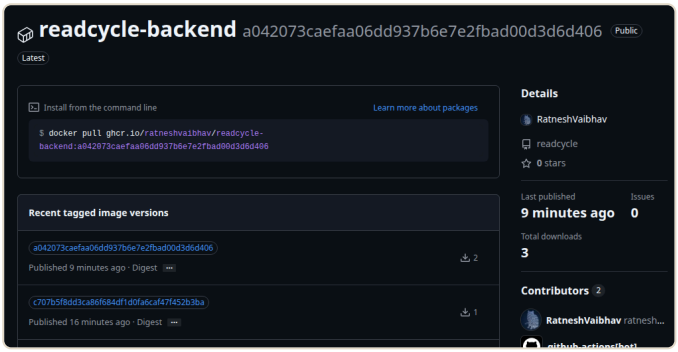
15 pts

`.github/workflows/pipeline.yml` runs on every push and pull request to `main`. No run has ever failed: 12 green so far, and 4 cancelled because a newer push superseded them.

Jobs, in order	What they do
Backend tests · Frontend build	pytest on SQLite and the Vite build; everything after them `needs` them
SAST · SCA · Secret scan	Bandit + Semgrep · pip-audit + npm audit + Trivy fs · Gitleaks over the whole history
Docker build · Image scan · Security gate	both images built once; Trivy scans those exact files; one policy over all 8 reports
Push · Deploy test · Promote	GHCR with the **commit SHA** as the tag (never `latest`) · kind + `helm test` · tag committed to `gitops/`



A green run on `main`: all 11 jobs passed. `web-m5-actions-run-green.png`



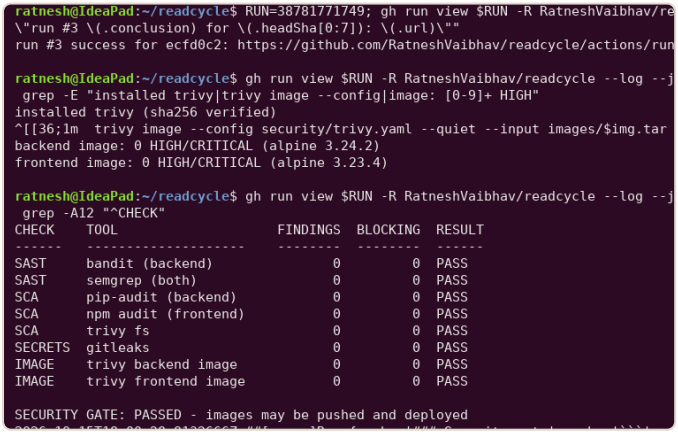
GHCR: `readcycle-backend`, public, tagged only with commit SHAs. `web-m5-ghcr-readcycle-backend.png`

M6 DevSecOps

5 pts

What Trivy scanned and what it means

Trivy scanned the operating-system packages and the Python and Node dependencies inside both images, using the exact files the pipeline pushes, and found **0 HIGH or CRITICAL**. That is deliberate: the upstream nginx base image has **43 fixable HIGH/CRITICAL CVEs**, for example **CVE-2026-33630**, a use-after-free / double-free in the `c-ares` DNS library (1.34.6, fixed in 1.34.8). The frontend Dockerfile runs `apk upgrade` and removes `curl` to patch them, and the gate fails the pipeline if one comes back. The scanners are pinned and verified with SHA-256 checksums.



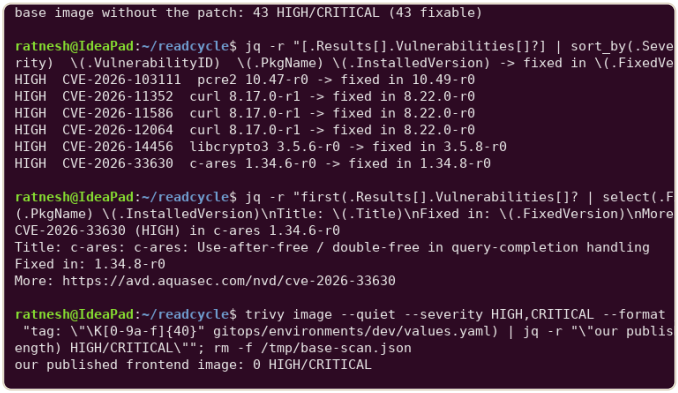
```
ratnesh@IdeaPad:~/readcycle$ RUN=38781771749; gh run view $RUN -R RatneshVaibhav/re
\"run #3 \(.conclusion) for \(.headSha[0:7]): \(.url)\""
run #3 success for ecfd0c2: https://github.com/RatneshVaibhav/readcycle/actions/run

ratnesh@IdeaPad:~/readcycle$ gh run view $RUN -R RatneshVaibhav/readcycle --log --j
 grep -E "installed trivy|trivy image --config|image: [0-9]+ HIGH"
installed trivy (sha256 verified)
^[[36;1m  trivy image --config security/trivy.yaml --quiet --input images/$img.tar
backend image: 0 HIGH/CRITICAL (alpine 3.24.2)
frontend image: 0 HIGH/CRITICAL (alpine 3.23.4)

ratnesh@IdeaPad:~/readcycle$ gh run view $RUN -R RatneshVaibhav/readcycle --log --j
 grep -A12 "^CHECK"
CHECK    TOOL                  FINDINGS  BLOCKING  RESULT
------   --------------------  --------  --------  ------
SAST     bandit (backend)             0         0  PASS
SAST     semgrep (both)               0         0  PASS
SCA      pip-audit (backend)          0         0  PASS
SCA      npm audit (frontend)         0         0  PASS
SCA      trivy fs                     0         0  PASS
SECRETS  gitleaks                     0         0  PASS
IMAGE    trivy backend image          0         0  PASS
IMAGE    trivy frontend image         0         0  PASS

SECURITY GATE: PASSED - images may be pushed and deployed
```

From the GitHub job logs: Trivy on both images (0 HIGH/CRITICAL) and the gate's table over all 8 reports: PASSED. `m6-trivy-in-pipeline.png`



```
base image without the patch: 43 HIGH/CRITICAL (43 fixable)

ratnesh@IdeaPad:~/readcycle$ jq -r "[.Results[].Vulnerabilities[]?] | sort_by(.Seve
rity)  \(.VulnerabilityID)  \(.PkgName) \(.InstalledVersion) -> fixed in \(.FixedVe
HIGH  CVE-2026-103111  pcre2 10.47-r0 -> fixed in 10.49-r0
HIGH  CVE-2026-11352  curl 8.17.0-r1 -> fixed in 8.22.0-r0
HIGH  CVE-2026-11586  curl 8.17.0-r1 -> fixed in 8.22.0-r0
HIGH  CVE-2026-12064  curl 8.17.0-r1 -> fixed in 8.22.0-r0
HIGH  CVE-2026-14456  libcrypto3 3.5.6-r0 -> fixed in 3.5.8-r0
HIGH  CVE-2026-33630  c-ares 1.34.6-r0 -> fixed in 1.34.8-r0

ratnesh@IdeaPad:~/readcycle$ jq -r "first(.Results[].Vulnerabilities[]? | select(.F
(.PkgName) \(.InstalledVersion)\nTitle: \(.Title)\nFixed in: \(.FixedVersion)\nMore
CVE-2026-33630 (HIGH) in c-ares 1.34.6-r0
Title: c-ares: c-ares: Use-after-free / double-free in query-completion handling
Fixed in: 1.34.8-r0
More: https://avd.aquasec.com/nvd/cve-2026-33630

ratnesh@IdeaPad:~/readcycle$ trivy image --quiet --severity HIGH,CRITICAL --format
 "tag: \"\K[0-9a-f]{40}" gitops/environments/dev/values.yaml) | jq -r "\"our publis
ength) HIGH/CRITICAL\""; rm -f /tmp/base-scan.json
our published frontend image: 0 HIGH/CRITICAL
```

The unpatched base image (43 fixable), CVE-2026-33630 in `c-ares`, and the published image: 0. `m6-trivy-cve-example.png`

M7 Terraform on AWS

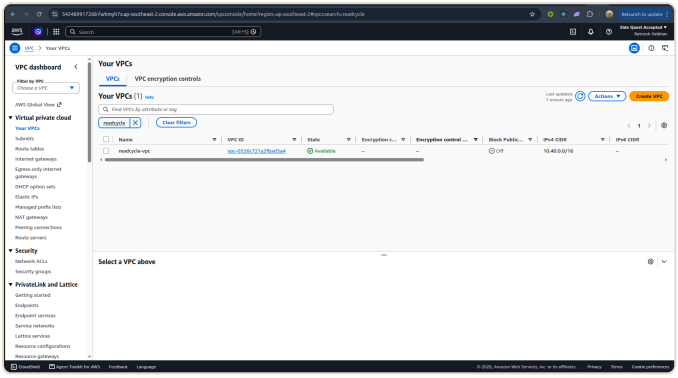
15 pts

`terraform/` creates **27 resources** (the first plan had 25; the Pod Identity fix added two):

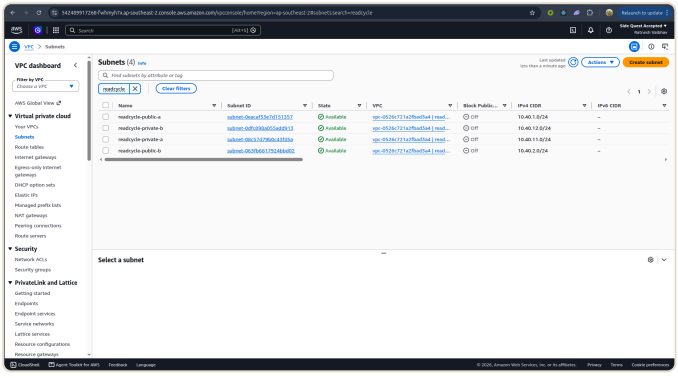
- a **VPC** `10.40.0.0/16` with **2 public** and **2 private subnets** in two availability zones;
- an internet gateway, a **NAT gateway**, route tables and IAM roles;
- **EKS 1.37** with a managed **node group** (one `m7i-flex.large`);
- the EBS CSI driver through **EKS Pod Identity**, and metrics-server.

Kept safe and free

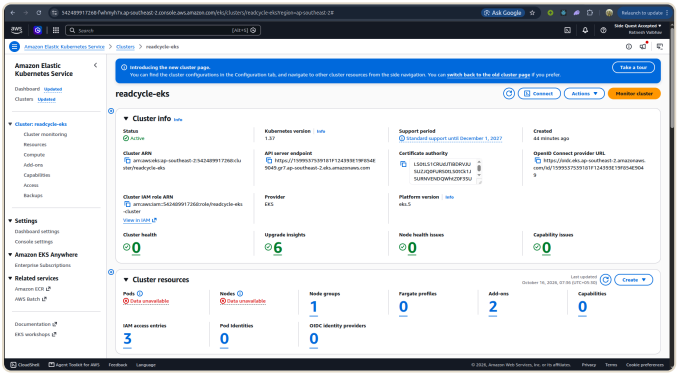
Sydney (ap-southeast-2): the account's AWS-managed policy allows EC2 and EKS only there. **Free plan:** `m7i-flex.large` is on its instance list. `upgrade_policy = STANDARD`: never the 6× pricier extended support. **No credentials in Git:** a least-privilege IAM user, its key only on the laptop (since deleted); `tfvars` and state git-ignored, `terraform.tfvars.example` committed. The EKS API accepted only the laptop's IP.



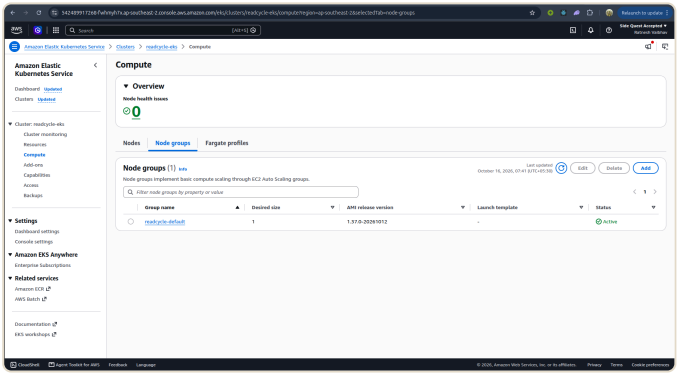
`readcycle-vpc` in Sydney. `aws-m7-console-vpc.png`



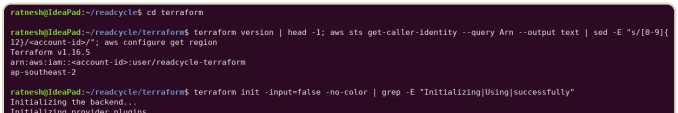
The four subnets, two public. `aws-m7-console-subnets.png`



EKS `readcycle-eks`: Active, 1.37, standard support. `aws-m7-console-eks-cluster.png`



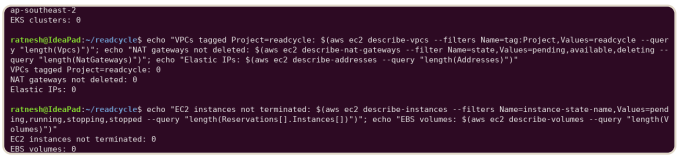
Node group `readcycle-default`: Active. `aws-m7-console-eks-nodegroup.png`



As the least-privilege user in ap-southeast-2: `init`, `fmt`, `validate`, `plan` (exit 0, 25 to add). `m7-terraform-plan.png`



`destroy`: 27 destroyed, 0 left in state. `m7-terraform-destroy_2.png`



Checked against AWS: no cluster, VPC, NAT gateway, Elastic IP, instance or volume left. `m7-aws-nothing-left.png`

Rubric criterion		Rubric criterion	
Valid HCL; `init` and a non-empty `plan`	✓	EKS with ≥ 1 node group, provisioned	✓
VPC with ≥ 2 public subnets, provisioned	✓	`destroy` removes everything; no credentials committed	✓

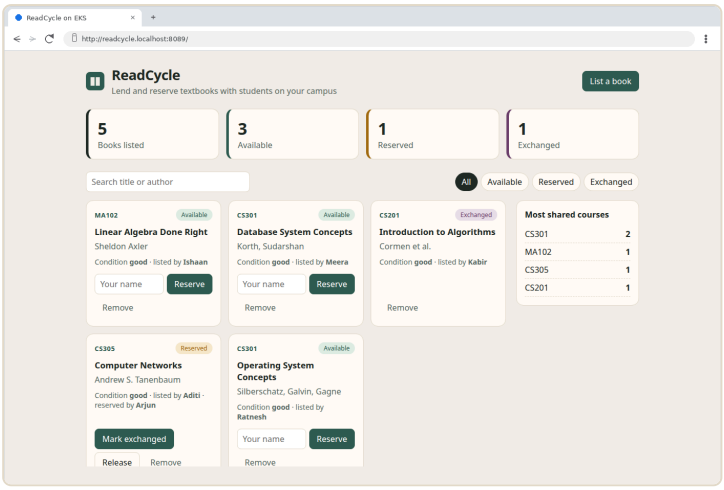
M8 Kubernetes + Helm

15 pts

The `helm/readcycle` chart, installed on **AWS EKS** and on the local cluster:

- **Backend**: HPA 2–4 replicas, startup/readiness/liveness probes, a `wait-for-db` init container, a ConfigMap, the password from a Secret.
- **Frontend**: 2 replicas. **PostgreSQL**: StatefulSet with a persistent volume (gp3 EBS).
- **ClusterIP Services** and an **Ingress**: `/` → frontend, `/api` → backend.
- ServiceMonitor, 4 alert rules, a Grafana dashboard, `helm test`; `k8s/namespace.yaml` with Pod Security labels.

> No AWS load balancer on EKS, because it would cost money: ingress-nginx is reached through `kubectl port-forward`, and requests still go through the Ingress rules by hostname.



ReadCycle on AWS EKS through the Ingress. `web-m8-eks-app-via-ingress.png`

```
ratnesh@IdeaPad:~/readcycle$ helm list -n readcycle
NAME            NAMESPACE       REVISION        UPDATED                                 STATUS          CHART           APP VERSION
readcycle       readcycle       1               2026-10-16 07:53:09.32753349 +0530 IST  deployed        readcycle-1.0.2 1.0.0

ratnesh@IdeaPad:~/readcycle$ kubectl get pods -n readcycle -o wide | cut -c1-140
NAME                                READY   STATUS    RESTARTS   AGE   IP             NODE                                                NO
readcycle-backend-7fc4d48d95-l4s88    1/1     Running   0          44s   10.40.12.175   ip-10-40-12-135.ap-southeast-2.compute.internal   <n
readcycle-backend-7fc4d48d95-zkfdp    1/1     Running   0          45s   10.40.12.240   ip-10-40-12-135.ap-southeast-2.compute.internal   <n
readcycle-frontend-5f88bf4599-k9f4f   1/1     Running   0          45s   10.40.12.195   ip-10-40-12-135.ap-southeast-2.compute.internal   <n
readcycle-frontend-5f88bf4599-t4xs4   1/1     Running   0          45s   10.40.12.59    ip-10-40-12-135.ap-southeast-2.compute.internal   <n
readcycle-postgres-0                  1/1     Running   0          44s   10.40.12.99    ip-10-40-12-135.ap-southeast-2.compute.internal   <n

ratnesh@IdeaPad:~/readcycle$ kubectl get svc -n readcycle
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
readcycle-backend    ClusterIP   172.20.139.82   <none>        8000/TCP   47s
readcycle-frontend   ClusterIP   172.20.212.30   <none>        80/TCP     47s
readcycle-postgres   ClusterIP   None            <none>        5432/TCP   47s

ratnesh@IdeaPad:~/readcycle$ kubectl get ingress,hpa,pvc -n readcycle
NAME                                CLASS   HOSTS                ADDRESS          PORTS   AGE
ingress.networking.k8s.io/readcycle   nginx   readcycle.localhost   172.20.121.180   80      48s

NAME                                                  REFERENCE                      TARGETS             MINPODS   MAXPODS   REPLICAS   A
GE
horizontalpodautoscaler.autoscaling/readcycle-backend   Deployment/readcycle-backend   cpu: <unknown>/70%   2         4         2          5
0s

NAME                                        STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS
   VOLUMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/data-readcycle-postgres-0   Bound    pvc-03c9fd81-f600-411e-8d9a-7fcd4dac2c7a   1Gi        RWO            gp3
   <unset>                 50s

ratnesh@IdeaPad:~/readcycle$ helm test readcycle -n readcycle | sed -n "/TEST SUITE/,\$p"
TEST SUITE:     readcycle-test-api
Last Started:   Fri Oct 16 07:54:13 2026
Last Completed: Fri Oct 16 07:54:20 2026
Phase:          Succeeded
```

On EKS: `helm list` (deployed), five pods Running, **ClusterIP** services, the Ingress, the HPA (min 2, max 4), the PVC on **gp3**, and `helm test` Succeeded. `m8-eks-helm-install.png`

```
ratnesh@IdeaPad:~/readcycle$ kubectl describe ingress readcycle -n readcycle | sed -n "/^Ru
Rules:
  Host               Path  Backends
  ----               ----  --------
  readcycle.localhost
                     /api   readcycle-backend:http (10.244.0.19:8000,10.244.0.21:8000)
                     /      readcycle-frontend:http (10.244.0.18:8080,10.244.0.20:8080)
Annotations:         meta.helm.sh/release-name: readcycle

ratnesh@IdeaPad:~/readcycle$ curl -s -o /dev/null -w "GET  /          -> HTTP %{http_code}
localhost:8088/; curl -s -o /dev/null -w "GET  /api/books  -> HTTP %{http_code} %{content_t
8/api/books
GET  /          -> HTTP 200 text/html  (frontend)
GET  /api/books  -> HTTP 200 application/json  (backend)
```

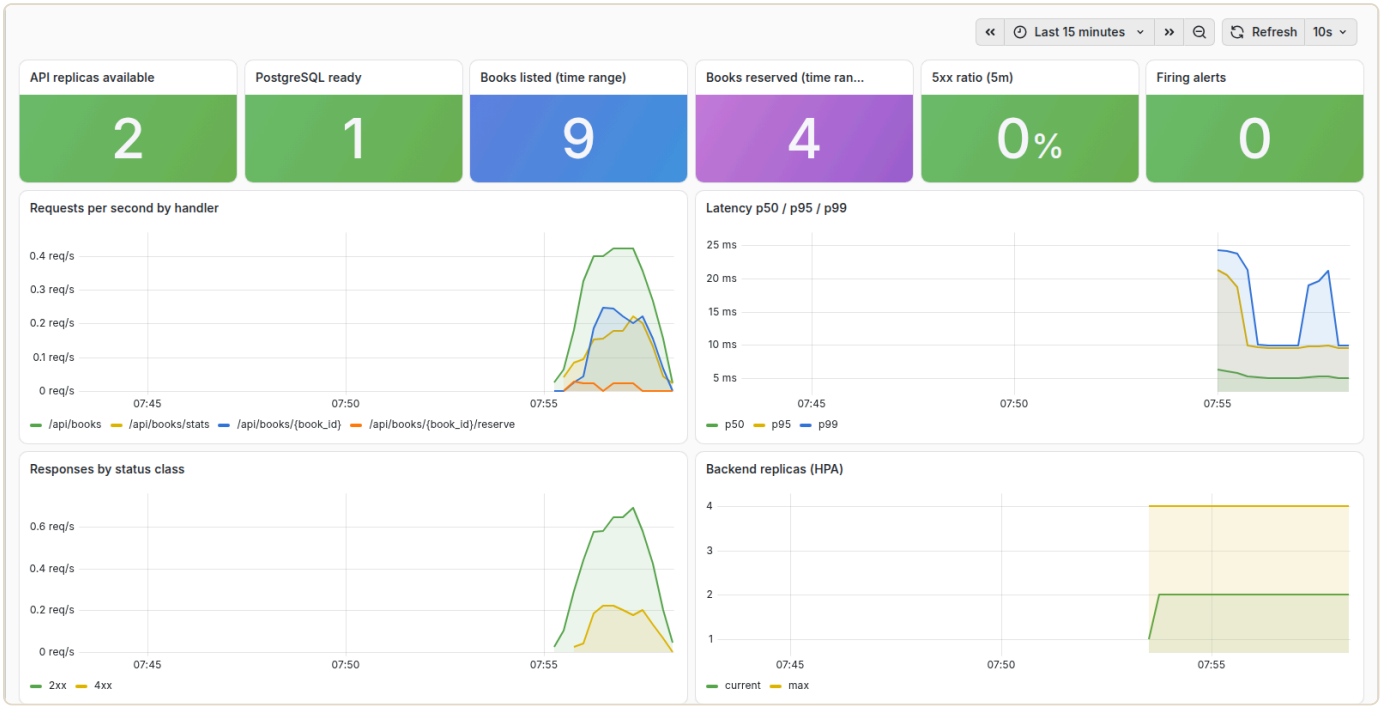
The Ingress rules: `/api` → backend, `/` → frontend, both answering HTTP 200. `m8-ingress-routing.png` (EKS: `m8-eks-ingress.png`)

Rubric criterion	
`namespace.yaml` applies; chart with `Chart.yaml`, values, templates	✓
`helm upgrade --install` works; `helm list`	✓
Backend and frontend ≥ 2 replicas, Running	✓
ClusterIP Services; Ingress `/` → frontend, `/api` → backend	✓
HPA scales 2 → 4 under load (page 8)	✓

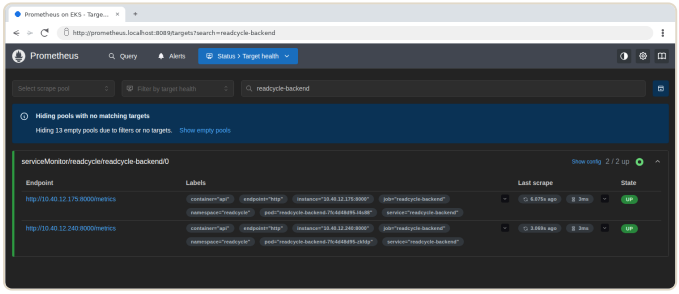
M9 Observability

10 pts

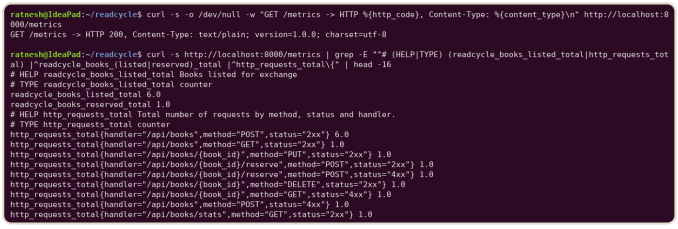
`monitoring/install.sh` installs kube-prometheus-stack with `monitoring/kube-prometheus-stack-values.yaml` (Prometheus on a volume, Grafana's admin password random in a Secret). The backend exposes `/metrics` through `prometheus-fastapi-instrumentator`; the chart's **ServiceMonitor** scrapes it every 15 s; **4 alert rules** watch for backend down, database down, 5xx ratio and p95 latency; a **12-panel dashboard** shows requests, latency, status codes, replicas, CPU and memory.



Grafana on AWS EKS with live traffic: replicas, books, 5xx ratio, alerts, requests per endpoint, latency p50/p95/p99, status codes, HPA replicas. `web-m9-eks-grafana-dashboard.png`



Prometheus on EKS: the backend targets UP. `web-m9-eks-prometheus-targets-up.png`



`/metrics`: HTTP 200, Prometheus text format, the app's own counters. `m9-metrics-curl.png`

```
ratnesh@IdeaPad:~/readcycle$ curl -s http://prometheus.localhost:8088/api/v1/rules | jq -r ".data.groups[] | select(.name==\"readcycle.alerts\") | .rules[] | \"\(.name)  state=\(.state)  health=\(.health)\""
ReadCycleBackendDown  state=inactive  health=ok
ReadCycleDatabaseDown  state=inactive  health=ok
ReadCycleHighErrorRate  state=inactive  health=ok
ReadCycleHighLatency  state=inactive  health=ok

ratnesh@IdeaPad:~/readcycle$ until [ "$(kubectl get hpa readcycle-backend -n readcycle -o jsonpath="{.status.desiredReplicas}")" -gt 2 ]; do
 sleep 5; done; date +%T; kubectl get hpa -n readcycle; kubectl describe hpa readcycle-backend -n readcycle | grep SuccessfulRescale | tail
-1 | cut -c1-150
23:27:19
NAME              REFERENCE                     TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
readcycle-backend   Deployment/readcycle-backend   cpu: 350%/70%   2         4         4          3m26s
  Normal   SuccessfulRescale            27s                 horizontal-pod-autoscaler  New size: 4; reason: cpu resource utilization (percentage o
```

The 4 alert rules loaded and healthy, and the HPA under load: CPU 350 % of the request, backend scaled from 2 to 4 replicas. `m9-promql-and-hpa.png`

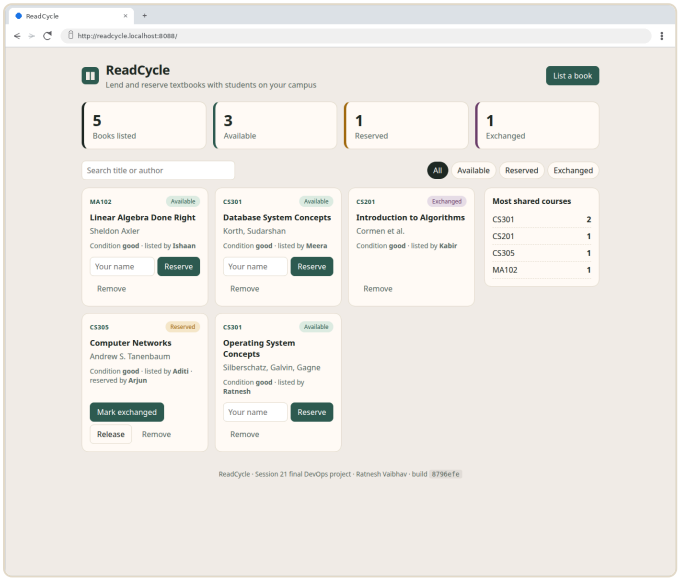
Rubric criterion		Rubric criterion	
`/metrics` returns Prometheus format	✓	Grafana accessible, a panel with live app metrics	✓
Prometheus installed and scraping the app	✓	`monitoring/` Helm values files	✓

M10 Documentation and live demo

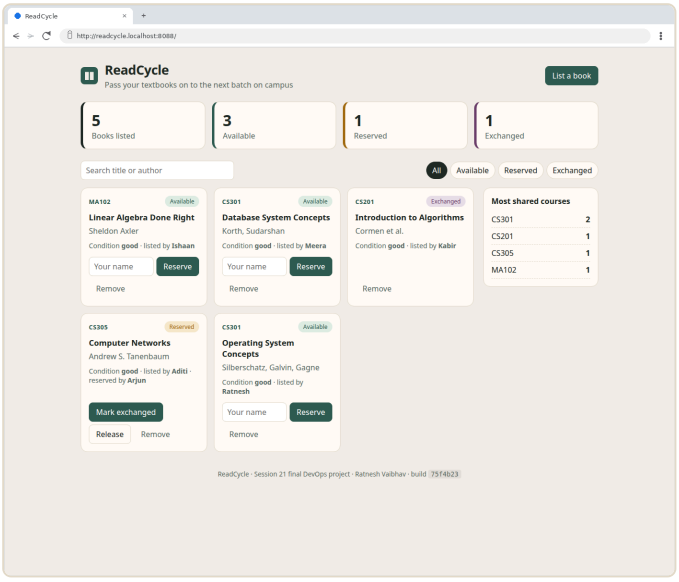
5 pts

`README.md` explains the application, the architecture and every part of the delivery system; `docs/DEMO.md` is the presentation script; `docs/EVIDENCE.md` maps every rubric item to its proof. `gitops/bootstrap.sh` prepares a cluster in one command: namespace, random database password, Argo CD, and the Application that deploys `helm/readcycle` from this repository's `main` (automated sync, prune, self-heal).

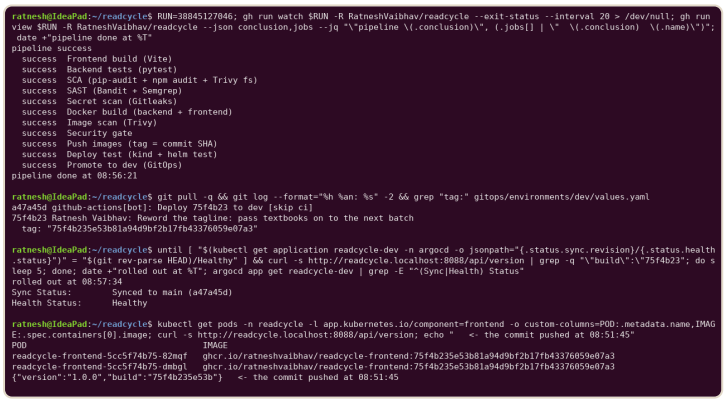
Time	The live demo, rehearsed and recorded: one commit to production
`08:51:45`	a tagline change is committed and pushed
`08:56:21`	all 11 pipeline jobs green; the promote job commits the new image tag to `gitops/`
`08:57:34`	Argo CD has rolled it out: the new tagline is live and the footer shows build `75f4b23`. Nobody ran `kubectl`.



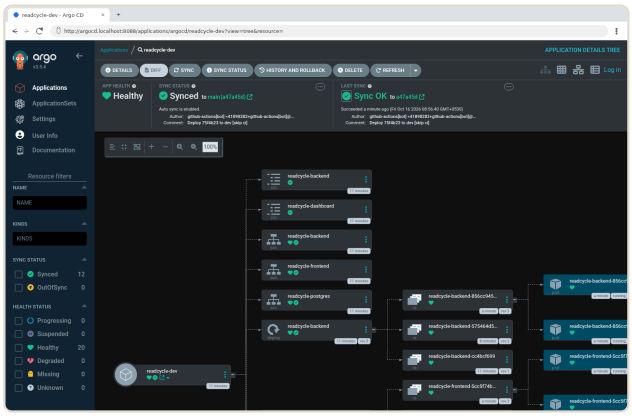
Before: the old tagline, build `8796efe`. `web-m10-1-before.png`



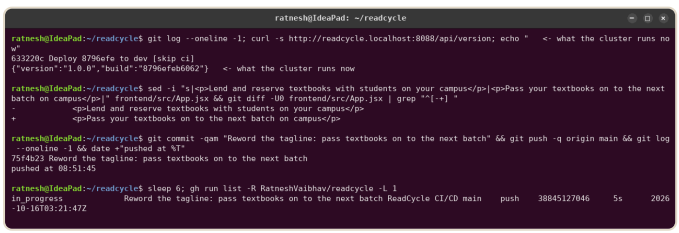
After: the new tagline, build `75f4b23`. `web-m10-3-after.png`



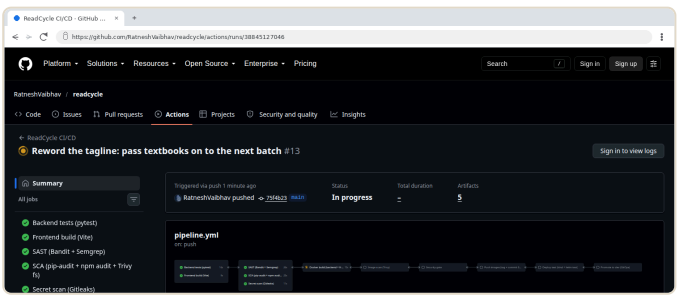
Pipeline success, the bot's tag commit, Argo CD synced and the new image running. `m10-live-demo-2-rollout.png`



Argo CD: Synced to the pipeline's commit, Healthy. `web-m10-4-argocd-synced.png`



The change: commit and push at 08:51:45. `m10-live-demo-1-commit.png`



The pipeline running for that commit. `web-m10-2-pipeline-running.png`

Troubleshooting challenge

Every fault arrived the way real outages do: **as a commit**. `troubleshooting/inject.sh` pushed a small, harmless-looking change to the GitOps values file and Argo CD deployed it. Each was identified, investigated through logs and resources, root-caused, fixed with `git revert` and verified. Full write-up: `troubleshooting/README.md`.

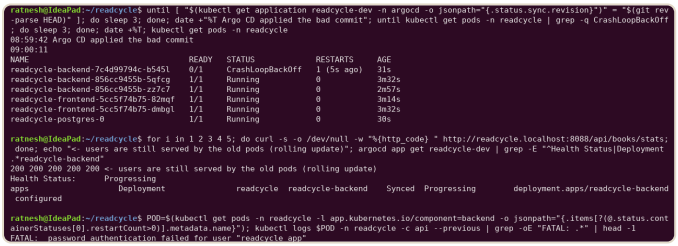
Bad commit	What users and tools saw	Root cause	Broken → fixed
`database.user: readcycle_app`	new backend pods crash-loop; users still served by old pods	PostgreSQL creates the user from `POSTGRES_USER` only on an empty volume	`08:59:42 → 09:00:31`
`ingress.host: readcycle.local`	**every URL 404 while Argo CD and all probes say Healthy**	no Ingress rule matches the hostname users type	`09:02:04 → 09:03:17`
`backend.resources: null`	HPA `cpu: <unknown>`, Argo CD Degraded	HPA utilisation is a % of the CPU request; no request, nothing to divide by	`09:04:41 → 09:06:28`

```
ratnesh@IdeaPad:~/readcycle$ troubleshooting/inject.sh troubleshooting/02-ingress-host-typo.values.yaml && git show --format= HEAD | tail -n
 +5
61c4873 Fault 2 - the ingress host is changed to a name nobody uses.
@@ -9,3 +9,5 @@ backend:
   env:
     APP_ENV: development
     LOG_LEVEL: INFO
+ingress:
+  host: readcycle.local

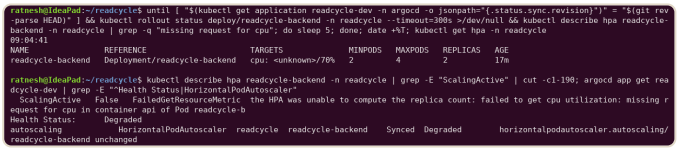
ratnesh@IdeaPad:~/readcycle$ until [ "$(kubectl get application readcycle-dev -n argocd -o jsonpath="{.status.sync.revision}/{.status.health
.status}")" = "$(git rev-parse HEAD)/Healthy" ] && [ "$(kubectl get ingress readcycle -n readcycle -o jsonpath="{.spec.rules[0].host}")" = r
eadcycle.local ]; do sleep 3; done; date +%T; argocd app get readcycle-dev | grep -E "^(Sync|Health) Status"; echo "^ Argo CD and every prob
e say all is well..."
09:02:04
Sync Status:        Synced to main (61c4873)
Health Status:      Healthy
^ Argo CD and every probe say all is well...

ratnesh@IdeaPad:~/readcycle$ curl -s -o /dev/null -w "UI   http://readcycle.localhost:8088/          -> HTTP %{http_code}\n" http://readcycl
e.localhost:8088/; curl -s -o /dev/null -w "API  http://readcycle.localhost:8088/api/books -> HTTP %{http_code}\n" http://readcycle.localhos
t:8088/api/books
UI   http://readcycle.localhost:8088/          -> HTTP 404
API  http://readcycle.localhost:8088/api/books -> HTTP 404
```

Fault 2: the bad commit, Argo CD Synced and Healthy, and still HTTP 404 for every user. `tr2-ingress-host_1.png`



Fault 1: a new pod in CrashLoopBackOff, users still served by the old pods, and the log: `password authentication failed for user "readcycle_app"`. `tr1-wrong-db-user.png`



Fault 3: HPA `cpu: <unknown>/70%`, "missing request for cpu", Argo CD Degraded. `tr3-hpa-without-requests.png`

The method, the same every time

Symptom (`curl` , `argocd app get`) → resources (`kubectl get` , `describe` , events) → logs (`--previous` , the ingress controller) → the wrong object → the commit (`git log`) → `git revert` → check the symptom again.

Also documented: six real problems

AWS refusing `aws login` ; the one-region account policy; the node group's missing service-linked role; the EBS driver without credentials (fixed with Pod Identity); the PostgreSQL image versus Pod Security; Argo CD's brief "Degraded" after a rollout.

Lessons learned

1. **Healthy isn't working.** The Ingress typo passed every probe and Argo CD check while every user got a 404. Something has to test the public URL.
2. **GitOps makes every change reviewable and reversible.** Faults, fixes and deployments are all commits; a rollback is `git revert`.
3. **Scan what you ship.** The base image carried 43 fixable CVEs that `docker build` happily produced.
4. **Pods shouldn't borrow the node's identity;** EKS Pod Identity gives each workload its own credentials. **The HPA divides by the request:** no CPU request, no autoscaling.
5. **Tear down what Terraform doesn't know about first.** Kubernetes created the EBS volumes, so `terraform destroy` alone would have left them behind.